Item 2 - Practice - Tertiary

Reminders

Typing text responses

Where you are instructed to type responses, use the files specified in the question text.

The margin characters, '>', indicate the maximum number of statements you can type.

Type at least nearly as many points as there are '>' characters to maximize your score.

Aim to keep statements brief and direct; prefer bullet points to longer sentences.

Case-sensitive by default

Unless stated otherwise, assume all instructions/examples are case-sensitive.

This means that 'A' is not the same as 'a'.

This can be relevant when referring to elements of source code or design parameters.

Text-Editing tools

You can open a text file in nano/vim by typing `nano <file>`/`vim <file>` in the terminal.

To open a file in kate via the terminal, type: `kate <file>`.

You can also open it by using the graphical application launcher in the corner.

Writing code

Canberra College IT Faculty

Aim to write the shortest amount of code which fulfills the specification.

Your code may be marked for readability and style.

Read the commented header of the response file for advice.

Canberra College IT Faculty

This is a practice paper

Some marking schemes are visible to inform your approach and response.

This item can only give you an idea of the format and tone of the actual item.

You should complete your class-based practice tasks and request feedback if necessary.

Reminders (Technology-Specific)

Basic Bash Commands Cheat Sheet

- `pwd` – Print the current working directory
- `ls` – List files and directories in the current location
- `cd <dir>` – Change to directory `<dir>`
- `mkdir <dir>` – Create a new directory named `<dir>`
- `rm <file>` – Remove a file
- `cp <src> <dest>` – Copy a file from `<src>` to `<dest>`
- `mv <src> <dest>` – Move or rename a file from `<src>` to `<dest>`
- `cat <file>` – Display the contents of a file
- `clear` – Clear the terminal screen

Executing Java programs

To compile and run your Java program, `YourFile.java`, from the terminal:

1. Navigate to the directory containing your `YourFile.java`
2. Compile it with `javac YourFile.java`
3. Run the compiled program with `java YourFile`

Canberra College IT Faculty

Working with Eclipse (Java IDE)

To create, edit, and run a simple Java program in Eclipse:

1. Open Eclipse and create a new Java Project via `File > New > Java Project`
2. Create a new Java class via `File > New > Class`. Include a `public static void main(String[ args)` method stub if you wish.
3. Type your program code into the editor.
4. Save your file with `Ctrl+S`.
5. Run the program with `Ctrl+F11` or via `Run > Run`.

Canberra College IT Faculty

Canberra College IT Faculty

Java documentation help

You do not need to memorize Classes!

- Hover your mouse cursor over any standard class to read Java documentation about it.
- You can view Java documentation in a separate pane: `Window > Show View > Javadoc`

Canberra College IT Faculty

Canberra College IT Faculty

Submitting Java files

Canberra College IT Faculty

Named submission files

Save your responses to the provided, named files, for example `Question1.java`, unless the question says otherwise.

Only those named files will be collected from your machine.

If a required file is renamed, moved out of its `Item` directory, or cannot be opened by markers, you risk a zero for that question or at least significant penalties.

Your responsibility: correct name, correct location, compilable content.

Provided class stub

For convenience, you may be provided a class stub with an empty `main` method.

You can compile and run this as-is, then add and run your own code.

Make sure to comment out any test code before submission.

```
public class Question1 {
    public static void main(String[] args) {
        // This stub has been added for your convenience
    }
}
```

Defining an additional required class in the same file

If a question requires you to define a specific class in the same file, for example `Foo`, make it package-private.

That means using no access modifier, so the public class name still matches the filename.

Example:

```
public class Question1 {
    public static void main(String[] args) {
        // This stub has been added for your convenience
    }
}

class Foo {
```

```
    private int bar;
    public String baz;
}
```

Canberra College IT Faculty

Canberra College IT Faculty

Jump To

Q1 (2 parts) 7

Q2 (2 parts) 8

Q3 (2 parts) 9

Q4 (2 parts) 10

Q5 (2 parts) 11

Q6 (2 parts) 12

Canberra College IT Faculty

We wrote our own textbook, because there was no other text about space. We just started from what we knew.

— Katherine Johnson

Programs must be written for people to read, and only incidentally for machines to execute.

— Harold Abelson

Q1 (2 parts)

Canberra College IT Faculty



a) A map app displays several kinds of marker through one collection. Declare the Java interface `MapMarker` with `String label()`. Implement it in `ShopMarker` and `ParkMarker`, add one of each to an `ArrayList<MapMarker>`, and print every label. Your output should contain one label per line.

Save your source code in `Question1a.java`.

b) The map may later add station and event markers. Evaluate how the `MapMarker` design supports that change.

- Explain the contract implemented by `ShopMarker` and `ParkMarker`.
- Explain how the polymorphic collection reduces changes to the display loop.

Save your response in `q1_response.txt`.

Canberra College IT Faculty

Q2 (2 parts)



a) A repair form accepts priorities from 1 to 5. Write `int readPriority(Scanner input)` so it keeps prompting until the user enters an integer in that range. It must consume invalid text safely, provide distinct feedback for invalid text and an out-of-range number, and return the accepted priority.

Canberra College IT Faculty

Save your source code in `Question2a.java`.

b) Design validation for a repair form that accepts an asset code and a priority from 1 to 5.

- Specify precise validity rules for both fields.
- Explain how feedback and repeated input should handle each invalid case without losing scanner input.

Save your response in `q2_response.txt`.

Canberra College IT Faculty

Q3 (2 parts)



a) A media app creates thumbnails in the background while its interface remains responsive. Write a Java class `ThumbnailTask` with an integer `created` field and an inner `Runnable` that sets `created` to 3 and prints `Thumbnails ready`. Start it in a `Thread`, wait using `join()`, then print `created`. Expected output:

```
Thumbnails ready
3
```

Canberra College IT Faculty

Save your source code in `Question3a.java`.

b) The thumbnail worker needs access to state held by its enclosing media object. Explain a safe division of responsibility.

- Describe the worker's responsibility and the enclosing state it needs.
- Explain one concurrency hazard and justify a suitable mitigation.

Save your response in `q3_response.txt`.

Q4 (2 parts)



a) A library kiosk stores loan lengths in days. Write `int totalLongLoans(ArrayList<Integer> lengths)` to return the sum of values greater than `14`. Call it with `7, 21, 30, 14` and print the returned value. Expected output:

```
51
```

Save your source code in `Question4a.java`.

b) A library kiosk currently uses one method to filter overdue loans, calculate charges, and build an alert. Propose a decomposition.

- Define focused method responsibilities and the data passed between them.
- Evaluate how the proposed boundaries improve maintenance and isolated testing.

Save your response in `q4_response.txt`.

Canberra College IT Faculty

Canberra College IT Faculty

Q5 (2 parts)



a) A maintenance app tracks repair work. Implement a Java class `RepairJob` with private `identifier`, `description`, and `completed` fields and a constructor that creates an open job. Add `boolean complete()`, which changes the state only once, and `String status()`, which returns `OPEN` or `DONE`.

Save your source code in `Question5a.java`.

Canberra College IT Faculty

b) Explain how the `RepairJob` design protects valid state.

Canberra College IT Faculty

- Describe the role of encapsulation and constructor validation.
- Explain how `complete()` prevents a repeated transition and reports whether state changed.

Save your response in `q5_response.txt`.

Q6 (2 parts)



a) A parcel app maps each delivery speed to a number of days. Declare enum `Speed` with `STANDARD`, `EXPRESS`, and `SAME_DAY`. Write `int deliveryDays(Speed speed)` using a switch expression to return `5`, `2`, or `0`, then print the `EXPRESS` result.

Save your source code in `Question6a.java`.

Canberra College IT Faculty

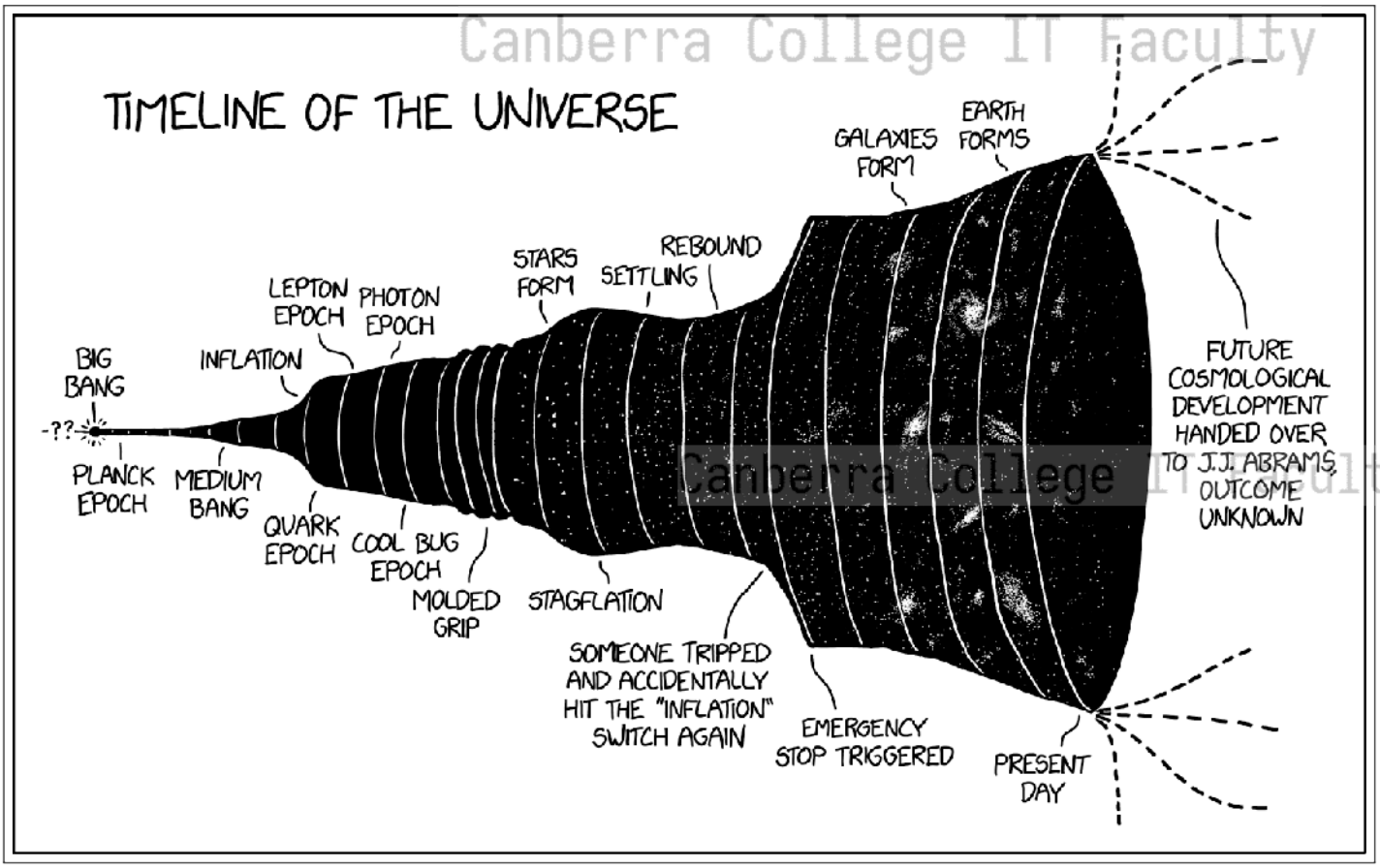
b) Evaluate the use of `Speed` rather than unrestricted text.

- Explain how the enum improves validity and readability.
- Explain how a complete switch expression helps maintain the program when a delivery speed is added.

Save your response in `q6_response.txt`.

Canberra College IT Faculty

Canberra College IT Faculty



“Timeline of the Universe”

— xkcd by Randall Munroe